Credit Card Fraud Detection

Using Machine Learning and Neural Networks

CSCI 611 Final Project

Jayana Sarma | Nayana Nagarajappa

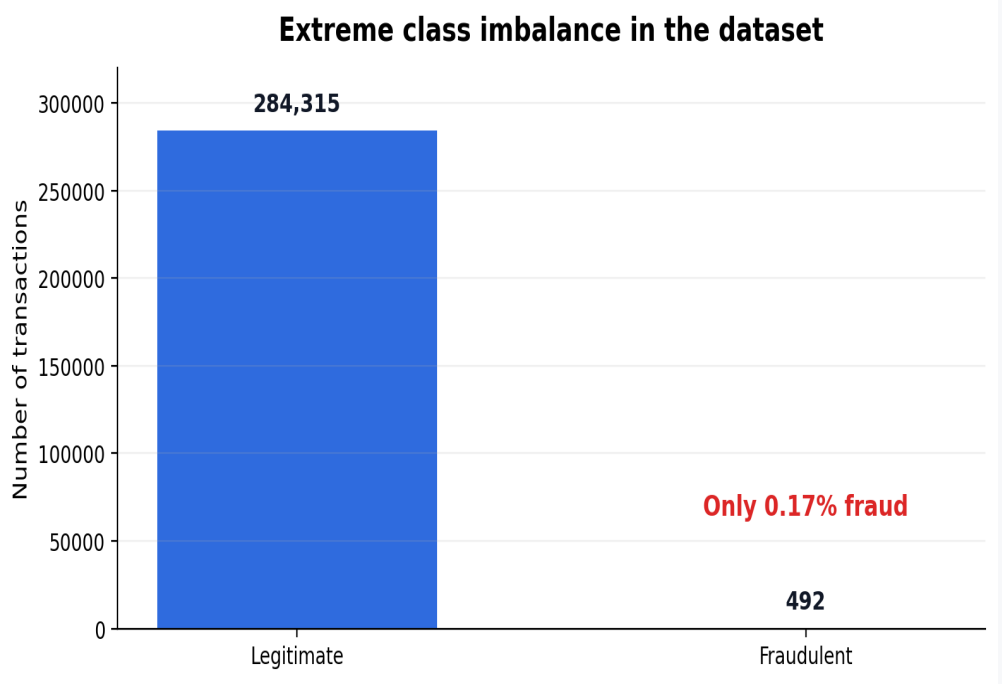
Application context

Fraud detection is a binary classification problem where accuracy alone can be misleading.

Why this problem matters

Fraudulent transactions are rare but costly.
A high-accuracy model can still miss most fraud.
The goal is to catch fraud while limiting false alarms.

Core challenge **Imbalanced classification**



Only 492 fraud cases out of 284,807 transactions in the original dataset.

Dataset and preprocessing

We used the Kaggle Credit Card Fraud Detection dataset. The dataset was prepared to keep training, testing, and app prediction consistent.

Input columns

Time, Amount, V1–V28

Target

Class: 0 = legitimate, 1 = fraud

Split strategy

Stratified train / validation / test

Preprocessing choices

- Checked class imbalance and feature structure
- Scaled numeric features for models that needed it
- Tuned thresholds on validation data

App-ready feature contract

The app loads the saved feature names and reorders uploaded CSV columns before prediction. This prevents training-serving mismatch.

Consistent preprocessing keeps the app aligned with the model training pipeline.

Key technologies and solution design

We compared baselines, tree-based models, boosting, and neural networks before selecting the final model.

Baseline

Logistic Regression
Balanced Logistic

Reference models

Tree-based

Random Forest
RF threshold 0.3

Best final model

Boosting

XGBoost

Strong ranking model

Neural networks

Feedforward NN
Deep NN + Dropout

Architecture comparison

Solution strategy

Use validation-based threshold tuning to balance fraud recall and false alarms, then report final performance on the test set.

Evaluation metrics

Because fraud cases are rare, accuracy is not enough to judge performance.

1

Precision

Of predicted fraud cases, how many were truly fraud?

2

Recall

Of actual fraud cases, how many did the model catch?

3

F1-score

Balance between precision and recall; used for final selection.

4

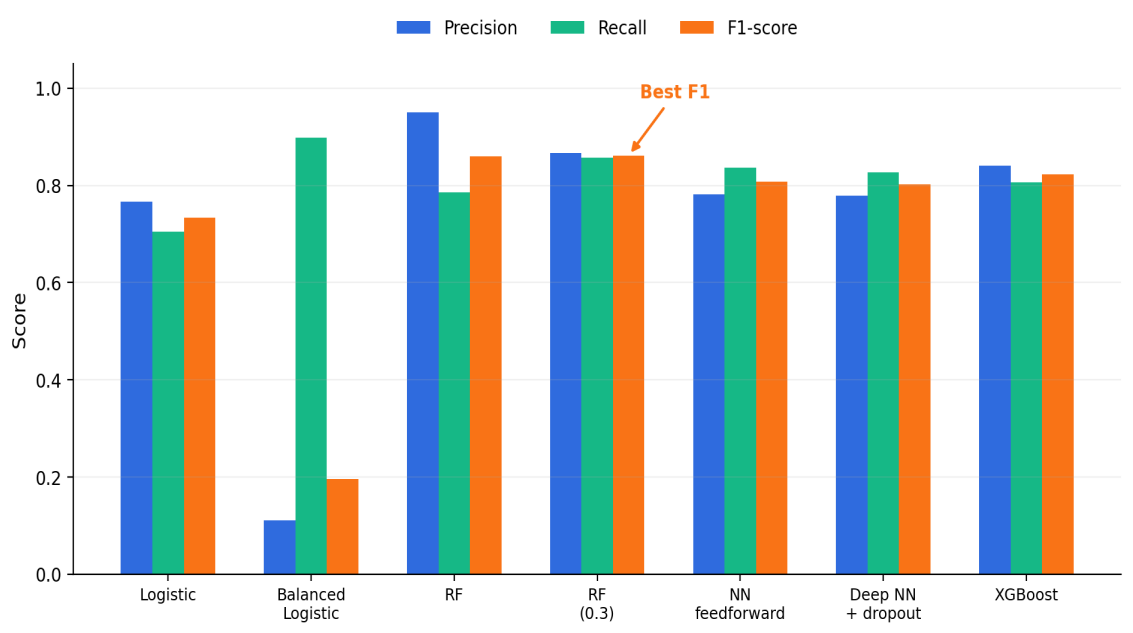
PR-AUC

Useful when the positive class is rare.

Critical tradeoff Higher recall catches more fraud; higher precision reduces false alarms; F1-score balances both.

Results: vertical progress

Model performance across the project stages, from baselines to tuned models.



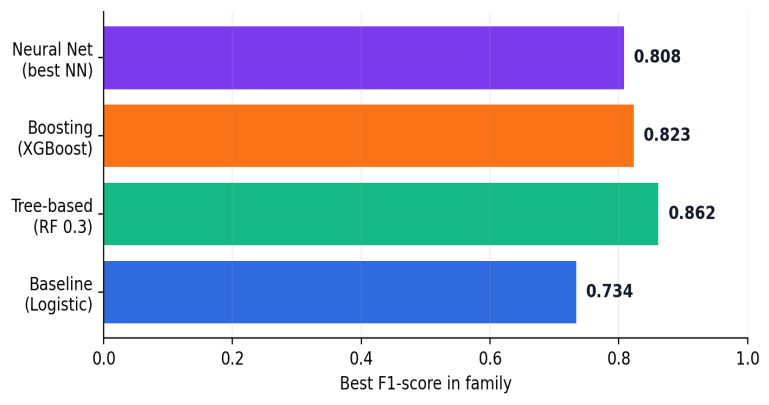
Key observations

Balanced Logistic had high recall but very low precision.
Tree-based models ranked highest overall.
NN models became competitive after architecture comparison.
RF threshold 0.3 achieved the best F1-score.

Best F1 **RF threshold 0.3**

Horizontal results: baseline vs advanced models

Reference baselines were compared against stronger model families.



Interpretation by model family

Baselines established a useful reference point.
Balanced Logistic caught many fraud cases but had too many false positives.
XGBoost performed strongly, but RF threshold tuning produced the best F1-score.
Neural networks were competitive but did not outperform tree-based models.

Final comparison takeaway: Random Forest with threshold 0.3 gave the best overall precision-recall balance.

Final model selection

Random Forest with a 0.3 threshold gave the best precision-recall balance.

Selected model

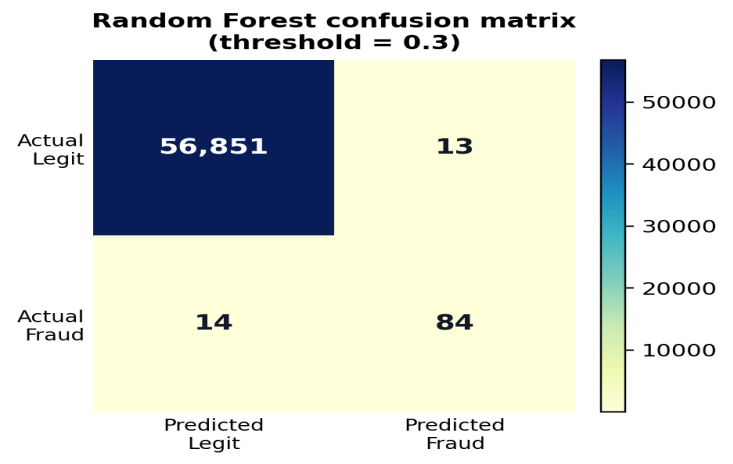
Random Forest

Threshold = 0.3

Why it was selected

Highest F1-score among tested models
Strong recall without too many false alarms
Works well on tabular transaction data
Easy to save and deploy in Streamlit

Precision	Recall	F1-score
0.8660	0.8571	0.8615



Interpretation: catches fraud while controlling false alarms.

Critical component: threshold-based prediction

The app converts model probabilities into fraud decisions using the selected threshold.

```
data_scaled =
scaler.transform(data_for_prediction)

fraud_probabilities =
model.predict_proba(data_scaled)[:, 1]

predictions = (fraud_probabilities >=
threshold).astype(int)
```

Why this matters

The model outputs a probability first. The threshold controls when that probability becomes a fraud alert, which directly affects precision and recall.

1 **Scale uploaded data**

2 **Predict probability**

3 **Apply threshold 0.3**

4 **Label transaction**

Decision rule **Probability ≥ 0.3 → Fraudulent | Probability < 0.3 → Legitimate**

App demonstration

The trained model was packaged into a simple app for CSV-based fraud prediction.

Demo flow

- Upload a 100-row sample CSV
- Run fraud prediction
- Review predicted fraud count
- Preview fraud cases
- Download full results

Limitations and future work

The current solution works as a project demo, with clear paths for improvement.

Current limitations

- Features are anonymized, so interpretation is limited
- App expects the same feature format as training data
- Model was trained on historical transactions only
- Fraud patterns can change over time

Future improvements

- Add real-time fraud detection support
- Monitor performance drift over time
- Try SMOTE or other resampling methods
- Use SHAP values for model explainability
- Deploy the app online for easier access

Main takeaway: fraud detection needs threshold-aware evaluation, not just accuracy.